Now, let's get perverse. The two-byte sequence \xcf\x80 represents π in UTF-8, but it is not a valid byte sequence in SJIS encoding. Let's see what happens if we tell Ruby that this same source file is SJIS encoded. (Remember, when we do this, we're not changing the actual bytes in the string—we're just telling Ruby to interpret them with a different set of encoding rules.)

```
# encoding: sjis
PI = "π"
puts "The size of a string containing π is #{PI.size}"
```

produces:

```
prog.rb: /tmp/prog.rb:2: invalid multibyte char (Windows-31J) (SyntaxError)
prog.rb:3: invalid multibyte char (Windows-31J)
```

This time, Ruby complains because the file contains byte sequences that are illegal in the given encoding.

Ruby supports a virtual encoding called ASCII-8BIT. Despite the *ASCII* in the name, this is really intended to be used on data streams that contain binary data. It's the default encoding that Ruby uses for reading binary streams. (It also has an alias of BINARY}.) However, you can also use this as an encoding for source files. If you do, Ruby interprets all characters with codes below 128 as regular ASCII and all other characters as valid constituents of variable names. This is basically a neat hack, because it allows you to compile a file written in an encoding you don't know—the characters with the high-order bit set will be assumed to be printable.

```
# encoding: ascii-8bit
π = 3.14159
puts "π = #{π}"
puts "Size of 'π' = #{'π'.size}"
```

produces:

```
π = 3.14159
Size of 'π' = 2
```

The last line of output illustrates why ASCII-8BIT is a dangerous encoding for source files. Because it doesn't know to use UTF-8 encoding, the π character looks to Ruby like two separate characters.

Converting Encodings

Strings, symbols, and regular expressions are labeled with their encoding. You can convert a string from one encoding to another using the encode method. For example, we can convert the word *olé* from UTF-8 to ISO-8859-1:

```
ole_in_utf = "olé"
ole_in_utf.encoding      # => #<Encoding:UTF-8>
ole_in_utf.bytes.to_a    # => [111, 108, 195, 169]

ole_in_8859 = ole_in_utf.encode("iso-8859-1")
ole_in_8859.encoding     # => #<Encoding:ISO-8859-1>
ole_in_8859.bytes.to_a   # => [111, 108, 233]
```

You have to be careful when using encode—if the target encoding doesn't contain characters that appear in your source string, Ruby will throw an exception. For example, the π character is available in UTF-8 but not in ISO-8859-1: